

A SHORT R TUTORIAL

Steven M. Holland

Department of Geology, University of Georgia, Athens, GA 30602-2501



26 September 2025

Installing R

R is open-source and freely available for macOS, Linux, and Windows. To download R, you have two main options. The first is to go to the CRAN (Comprehensive R Archive Network) site and click on one of the three links under “Download and Install R” that are specific to the three main operating systems; do not click on the links under “Source Code” unless you have a specific reason for compiling R yourself. The second is to install RStudio by going to the downloads page for Posit (<https://posit.co/downloads/>).

Regardless of how you download R, the R engine that runs in the code is *identical*. You can run R in many different ways, including the R application, RStudio, the terminal, Jupiter notebooks, and even in Excel. For that reason, describe your work as being done in R, not the application in which you run R (e.g., RStudio).

The R.app for macOS, available from CRAN, is quite good, but some prefer the RStudio interface. On Windows, RStudio is superior to the R.app.

In addition to R, you should install a good text editor for writing and editing code; do not use a word processor (like Word). BBEdit is an excellent text editor for macOS and is available from Bare Bones Software. For Windows, Notepad++ is highly recommended, and it is free. Sublime Text is another highly regarded text editor, and it runs on all three platforms.

Learning R

There is a burgeoning number of books on R, and I highly recommend these three.

Statistics: An Introduction using R, by Michael J. Crawley, 2014. John Wiley & Sons, 360 p. ISBN-13: 978-1118941096. A good combination of statistics and R; it hits the right balance that many other books miss.

The R Book, by Michael J. Crawley, 2012. Wiley, 1076 p. ISBN-13: 978-0470973929. This is my favorite book on R; it’s the first book I turn to when I have questions.

The Art of R Programming: A Tour of Statistical Software Design, by Norman Matloff, 2011. No Starch Press, 400 p. ISBN-13: 978-1593273842. A great guide if you have a background in programming or want to get serious about programming in R.

My interests lean towards ecology, and these next two are excellent.

Numerical Ecology with R, by Daniel Borcard, François Gillet, and Pierre Legendre, 2018. Springer, 435 p. ISBN: 978-3-319-71403-5. Hands-down, this is the best book on using R for ecological analysis.

Ecological Models and Data in R, by Benjamin M. Bolker, 2008. Princeton University Press, 408 p. ISBN-13: 978-0691125220. A good guide to using Bayesian statistics in R for ecological questions.

I don't cover tidyverse and ggplot in this tutorial, although I use them when needed. These next three are the best I've seen on the subject. All of them are fantastic.

R for Data Science, by Hadley Wickham, Mine Çetinkaya-Rundel, and Garrett Grolemund, 2023. O'Reilly Media, 548 p. ISBN: 978-1492097402. I don't cover tidyverse and ggplot in this tutorial, but this is the best place I've found to learn them.

R Graphics Cookbook, by Winston Chang, 2019. O'Reilly Media, 425 p. ISBN: 978-1-491-97860-3.

Data Visualization, by Kieran Healy, 2019. Princeton University Press, 272 p. ISBN: 978-0-691-18161-5. This is another great way to learn ggplot.

The manuals link on the CRAN page points to three useful guides. The *Introduction to R* is a primary source of information on R. *R Data Import/Export* helps understand how data may be imported into or exported from R. The *R Reference Index* is a gigantic PDF (3500 pages!) that comprehensively lists all help files in a standard R installation. These help files are also freely accessible in every installation of R.

Every experienced R user likely has their favorite websites for R, and these three are mine:

R-bloggers (<http://www.r-bloggers.com>) is a good news and tutorial site that aggregates hundreds of contributors. Following its RSS feed is an excellent way to stay on top of what's new and discover new tips and analyses.

Cookbook for R (<http://www.cookbook-r.com>) has recipes for working with data. This is a good source for how to do everyday operations.

R Graph Gallery (<https://r-graph-gallery.com/>) is a library of beautiful plots with recipes for making them. It's a great place to get inspired.

R Charts (<https://r-charts.com/>) is another beautiful and inspirational compilation of charts you can make in R. Their visual guide to colors in R is outstanding.

Stack Overflow (<https://stackoverflow.com/collectives/r-language>) is a question-and-answer site for programmers. Users post questions, other users post answers, and these get voted up or down by the community, so you can see what is regarded as the best answer and alternative approaches. For this reason, Stack Overflow is the first site I turn to when I have questions.

When you run into a problem, remember that **Web search is your friend**. AI is getting better, but I cannot yet recommend it; if you are learning R for the first time, AI is likely to become a crutch that will slow your learning.

Objects and Arithmetic

When you launch R, you will be greeted with a prompt (`>`) and a blinking cursor:

```
>
```

I will show the prompt for every command in this tutorial, but you should not type it.

R works with objects, and there are many types. Objects store data, and they have commands that can operate on them. A single number or text string is the simplest object, and it is called a vector.

To give a value to an object, use one of the two assignment operators. Although the equals sign may be more familiar to you now, the arrow (less-than sign, followed by a dash: `<-`) is more common, and you should use it.

```
> x = 3
> x <- 3
```

Read both these lines as “assign the value of 3 to the variable x” or “assign 3 to x”. We assign values and results to operators so that we can use them. To display the value of any object, type its name at the prompt.

```
> x
```

If you have used Excel, you will discover a fundamental difference between it and R: In Excel, you see the results, but the formula is hidden, and this is reversed in R.

Arithmetic operators use the standard symbols for addition, subtraction, multiplication, division, and exponents:

```
> x <- 3 + 4
> x <- 2 - 9
> x <- 12 * 8
> x <- 16 / 4
> x <- 2 ^ 3
```

Operators follow the standard order of operations: parentheses and brackets first, followed by exponents, then multiplication and division, with addition and subtraction evaluated last. Within each of these, operations proceed from left to right. Although you can always force an order of operations with parentheses, learn this order so that you use the fewest parentheses. Excess parentheses make code harder to read.

Comments are preceded by a pound sign (`#`), and what follows the pound sign on that line will not be executed.

```
> # x <- 2 * 5; everything on this line is a comment
> x <- 2 * 5 # comments can also be placed after a statement
```

Spaces are generally ignored, but include them to make code easier to read. Both of the following lines produce the same result.

```
> x<-3+7/14
> x <- 3 + 7 / 14
```

R is case-sensitive, so capitalization matters.

```
> x <- 3
> x      # correctly displays the 3 stored in x
> X      # produces an error, as X doesn't exist, but x does
```

Because capitalization matters, avoid giving objects names that differ only in their capitalization. Use capitalization consistently. One common pattern is *camel-case*, in which the first letter is lowercase, but subsequent words are capitalized (for example, **pacificCarbon**). *Snake case* is another common pattern, where words in an object's name are separated with an underscore (**pacific_ocean_carbon**). Pick a convention and be consistent.

In addition, avoid overly short or cryptic names for objects; it is better to have a longer name that is easily read because it will make your code easier to understand and debug. At the same time, overly long names are cumbersome to type, so strike a good balance between clarity and brevity.

Use the up and down arrows to reuse old commands without retyping them. The up arrow lets you access progressively older previously typed commands. Return to more recent commands using the down arrow. These commands can be edited, so the up and down arrows are good time-savers, even if you don't need to use precisely the same command as before.

Functions

Functions are a second type of object, and R has a rich set of functions. Functions are called with their *arguments* in parentheses, and they usually *return* a value. For example, the maximum of two or more values is calculated with the **max()** function:

```
> max(9, 5)
> max(x, 42) # objects can be arguments to functions
```

Some functions need no arguments, but the parentheses must still be included. If you forget them, R will interpret what you type as the name of a non-function object, like a vector or matrix. For example, the function **objects()**, which is used to display all R objects you have created, does not require any arguments and is called like this:

```
> objects()      # correct way to call a function
> objects        # error: this doesn't call the function
```

Neglecting the parentheses means that you are asking R to display the value of an object called **objects**. Since **objects** is a function, R will show you the source code for the function.

Functions commonly *return* a value. If the function is called without assigning the result to an object, the returned value will be displayed on the screen. If an assignment is made, the value will be assigned to that object, but it won't be displayed on the screen.

```
> max(9, 5)           # displays the result
> myMax <- max(9, 5)   # result is stored, not displayed
```

Multiple functions can be called in a single line, and functions can be nested so that the result of one function becomes an argument for another function. Nesting functions too deeply can make the command confusing and hard to debug.

```
> y <- (log(x) + exp(x * cos(x))) / sqrt(-cos(x) / exp(x))
```

To get help with any function, use the `help()` function or use the `?` operator, along with the name of the function.

```
> help(max)
> ?max
> ? max
```

The help pages show useful options for a function, background on how the function works, references to the primary literature, and examples that illustrate how the function could be used. Use the help pages.

In time, you will write your own functions because they let you invoke multiple commands with a single command. To illustrate how a simple function is created and used, consider the `pow()` function below, which raises a base to an exponent. In parentheses after the word **function** is a list of *parameters* to the function, that is, values that must be input into the function. When you call a function, you supply *arguments* (values) for these parameters. The commands the function performs are enclosed in curly braces. Indenting these statements is not required, but indentation makes the function easier to read.

```
> pow <- function(base, exponent) {
  result <- base^exponent
  result
}
```

Arguments can be assigned to a function's parameters in two ways: by name or position, and there are advantages and disadvantages to both. When you assign arguments *by name*, they can be listed in any order, and the function will give the same result:

```
> pow(base=3, exponent=2) # returns 9
> pow(exponent=2, base=3) # also returns 9
```

Note that assignment to arguments uses the `=` sign, not the `<-` you use when assigning to an object. You must use the argument names defined for the function; you will get an error if you don't. For example, don't use `exp` instead of `exponent` in the `pow()` function.

Assigning arguments *by position* involves less typing, but the arguments must be in the correct order.

```
> pow(3, 2) # returns 3^2, or 9
> pow(2, 3) # returns 2^3, or 8
```

For `pow()`, the first position is assumed to hold the first parameter in the function definition (**base**), and the second position is assumed to be the second parameter (**exponent**).

Calling arguments by position is faster, but calling arguments by name is less prone to error, and it makes the code self-explanatory.

Some functions have default values for parameters. For example, `pow()` could be written as

```
> pow <- function(base, exponent=2) {  
  result <- base^exponent  
  result  
}
```

Doing so makes the default exponent equal to 2. If you write `pow(5)`, you'll get 25 in return (5 to the default exponent of 2). If you want a different exponent, specify the **exponent** argument, such as `pow(2, 4)`, which would produce 16 (2 to the fourth power). Refer to a function's help page to see which function parameters have default values.

Several functions are useful for manipulating objects in R. To show all current objects, use `objects()` or `ls()`, which are different names for the same function.

```
> objects()  
> ls()
```

To remove objects, use `remove()`. Removing an object is permanent; it cannot be undone.

```
> remove(someObject)
```

You can remove all objects, but since this cannot be undone, be sure that this is what you want to do. Even if you remove all objects, your command history is preserved so that you can reconstruct your objects, but this is likely laborious. To remove all objects, you must combine two commands, the `ls()` function and the `remove()` function.

```
> remove(list=ls())
```

Vectors

Data can be stored in several different types of objects, and a vector is the most common. A vector is a series of values of the same type, such as numbers, characters (strings), and logical (Boolean). Vectors are created most easily with the `c()` function (*c* as in *concatenate*). Short vectors are easily entered this way, but long ones are more easily imported (see Importing Larger Data Sets below).

```
> x <- c(3, 7, -8, 10, 15, -9, 8, 2, -5, 7, 8, 9, -2, -4, -1)
```

Strings are enclosed in quotes. You can use single quotes or double quotes, but be consistent.

```
> y <- c('gold', 'copper', 'silver')
```

Logical values are either **TRUE** or **FALSE**, always uppercase. Although you can use T or F, this is discouraged because of the similarity of their letter shapes; it is easier to spot errors if the full word is used.

```
> y <- c(TRUE, TRUE, FALSE, TRUE, FALSE)
```

Vectors can be created in other ways, too, such as the colon (:) operator, which creates a series of sequential integers. Notice that `c()` is not used because `3:10` is already a vector.

```
> y <- 3:10
```

An element of a vector can be retrieved using an index, which describes its position in the vector, starting with 1 for the first element. The element is placed in square brackets after the name of the vector. To see the fifth element of `x`, type:

```
> x[5]          # returns 15
```

Multiple consecutive indices can be specified with a colon. For example, to retrieve elements 3 through 10, type

```
> x[3:10]       # returns -8, 10, 15, -9, 8, 2, -5, 7
```

To retrieve non-consecutive elements, create a vector of those indices using the `c()` function. Failing to use `c()` will cause an error.

```
> x[c(1, 3, 5, 7)] # returns 3, -8, 15, 8
```

These two approaches can be combined in more complex ways. For example, if you wanted elements 3 through 7, then elements 5 through 9, you would type:

```
> x[c(3:7, 5:9)]  
# returns a vector with -8, 10, 15, -9, 8, 15, -9, 8, 2, -5
```

You will often use conditional logic to select elements that meet certain criteria. For example, the following will return a logical vector stating whether each element is greater than zero.

```
> x>0
```

Any logical statement inside the square brackets will return the matching elements.

```
> x[x>0]        # all positive values in x  
> x[x<=2]       # values less than or equal to 2  
> x[x==2]       # all values equal to 2  
> x[x!=2]       # all values not equal to 2  
> x[x>0 & x<3]  # values greater than 0 and less than 3  
> x[x>5 | x<1]  # values greater than 5 or less than 1
```

Any Boolean vector can be negated with the `!` operator, which turns **FALSE** to **TRUE**, and vice versa. The `!` operator is easily overlooked when reading code. Except for `!=`, it is often better to use other operators that are more direct, for example, `x<2` rather than `!x>=2`. You will use logical operations frequently.

Vectors can be sorted with the `sort()` function. Ascending order is the default.

```
> sort(x)
```

Sorting can be done in descending order by changing one of the default arguments to `decreasing = TRUE` or reversing the sorted vector with `rev()`. There are often multiple ways to perform operations in R, and you should choose the simplest one.

```
> sort(x, decreasing=TRUE)
> rev(sort(x))
```

R speeds calculations by using vectorized math. For example, suppose you wanted to multiply all values in a vector by a constant. If you have programmed before, you might think of doing this with a loop, in which you step through each value in a vector and multiply it by a constant. Avoid doing this in R, as loops are slow, so much so that they won't be presented until near the end of this tutorial. Instead, multiply the vector by the constant in one line. The following example multiplies all of the values in a vector by 2.

```
> x <- c(1, 3, 5, 7, 9)
> 2 * x      # returns 2, 6, 10, 14, 18
```

Likewise, you can use vector arithmetic on two vectors. For example, given two vectors of the same length, it is easy to add the elements of one vector to the corresponding elements in a second vector, producing a third vector of the same length that contains the sums.

```
> x <- c(1, 3, 5, 7, 9)
> y <- c(2, 2, 2, 4, 4)
> x + y      # returns 3, 5, 7, 11, 13
```

Two vectors can be joined into a single longer vector with the `c()` function.

```
> z <- c(x, y)
```

The number of elements in a vector can be found with the `length()` function.

```
> length(x)
```

Factors

Factors are similar to vectors, but are limited to categorical (nominal) variables that have a fixed number of levels. For example, if you have a series of sites, each representing a specific habitat type, store them as a factor rather than as a vector:

```
> habitats <- factor(c('marsh', 'grassland', 'forest', 'tundra',
  'grassland', 'tundra', 'forest', 'marsh', 'grassland'))
```

Inspecting the `habitats` object shows that a factor is stored differently from a vector.

```
> habitats
[1] marsh      grassland forest      tundra      grassland tundra
[7] forest      marsh      grassland
```

```
Levels: forest grassland marsh tundra
```

The `str()` function (think “structure”) will display the internal structure of a factor or any other object, including vectors, lists, matrices, data frames, and even functions:

```
> str(habitats)
Factor w/ 4 levels "forest","grassland",...: 3 2 1 4 2 4 1 3 2
```

Although the names are displayed when we show the structure of a factor, the data are saved as values 1–4, each corresponding to a named factor. For example, the first element is saved as a 3, the third level of the factor (marsh). The second element is saved as a 2, the second level of the factor (grassland), and so on. This is not only a more compact way of saving the data, but it also allows efficient ways to use these categories, such as using `apply()` and `by()` to perform calculations on each category, `split()` to separate the data by category, and `table()` to create tables of the data by category.

Lists

Lists are another type of data structure, one that is closely related to vectors. Like vectors, lists hold a series of values, but the values in a list can be of different types. For example, the first element of a list might be a string giving the name of a locality ('**Wyoming**'), the second element might be an integer expressing the number of beetle species there (**42**), and the third element might be a boolean value stating whether the locality is an old-growth forest (**TRUE**). Lists are made with the `list()` function:

```
> x <- list('Wyoming', 42, TRUE)
```

Elements of a list are accessed with double brackets.

```
> x[[1]]           # returns the first item, 'Wyoming'
```

Statistical functions, such as regressions, commonly produce lists in which the elements are named, allowing them to be accessed by their names. For example, when you perform a linear regression, a linear regression `lm` object is returned, and it is a list that contains twelve items, including the slope, the intercept, confidence limits, *p*-values, residuals, etc. If you ran a regression and assigned the result to `forestRegression`, you could find the names of all the elements in the `forestRegression` list with the `names()` function.

```
> names(forestRegression)
```

One of the elements of the list is called `residuals`, and you can use *dollar-sign notation* to access it by name:

```
> forestRegression$residuals
```

You could also use double brackets and the name (in quotes) to access it, although it takes a bit more typing.

```
> forestRegression[['residuals']]
```

There's a third way to get the residuals. Because residuals are the second element in the list, you could also use double brackets with their position:

```
> forestRegression[[2]]
```

In each of these, the **residuals** object is a vector, so you can access individual elements of it by adding brackets to the end of any of these three commands. All three of these will return the third residual:

```
> forestRegression$residuals[3]
> forestRegression[['residuals']][3]
> forestRegression[[2]][3]
```

The first two methods are preferred because they make it clear that the residuals are being accessed, and the first method is the shortest.

Two or more lists can be joined into a single list with the **c()** function.

```
> longList <- c(listOne, listTwo)
```

Matrices

Another type of R data structure is a matrix, which has multiple rows and columns of values of the same type, like a vector. You can think of a matrix as a collection of vectors of the same length and type.

Small matrices are entered easily with the **matrix()** function, where you specify the data as a vector, followed by the number of rows or columns, using **nrow** or **ncol**.

```
> x <- matrix(c(3, 1, 4, 2, 5, 8), nrow=3)
```

This will generate a matrix of 3 rows and two columns since the matrix has six elements. By default, the matrix is filled by columns, such that column 1 is filled first from top to bottom, then column 2, etc. Displaying this matrix gives

```
      [,1] [,2]
[1,]     3     2
[2,]     1     5
[3,]     4     8
```

A matrix can alternatively be filled by rows by changing the default argument to be **byrow=TRUE**. Row 1 filled first, from left to right, then row 2, etc.

```
> x <- matrix(c(3, 1, 4, 2, 5, 8), nrow=3, byrow=TRUE)
```

This gives

```
      [,1] [,2]
[1,]     3     1
[2,]     4     2
[3,]     5     8
```

The bracketed values along the right and the top are the row and column positions. For example, the value in the third row and second column could be retrieved with

```
> x[3, 2]      # returns 8
```

Memorize this order: row first, column second. This is standard across R. If a matrix has column names (this one doesn't), values can also be accessed by name (remember the quotes):

```
> x[, 'columnName']
```

Column names and row names can be assigned to matrices, as described below for data frames.

Dollar-sign notation cannot be used on matrices. Attempting to do this will return a cryptically worded error about the \$ operator being invalid for atomic vectors.

```
> x$columnName
```

Matrices can also be made by making vectors and then joining them as columns using `cbind()` or as rows using `rbind()`. For example, we could make the same matrix in either of these ways:

```
> h <- c(3, 4, 5)
> j <- c(1, 2, 8)
> x <- cbind(h, j)

> h <- c(3, 1)
> i <- c(4, 2)
> j <- c(5, 8)
> x <- rbind(h, i, j)
```

The advantage of these is that they will automatically name the columns or the rows, and that may be useful to you.

Large matrices can be entered this way, but importing them is much easier and less error-prone (see Importing Larger Data Sets below).

Matrices use vectorized math, considerably speeding computation.

The rows and columns of a matrix can be swapped, such that the rows become the columns, and vice versa, by transposing the matrix with the `t()` function:

```
> y <- t(x)
```

Data Frames

A data frame is like a matrix, but the columns can be of different types. For example, one column might be locality names (strings), other columns might have numeric counts of different species of beetles, and other columns might be logical values that describe properties of the localities, such as whether it is an old-growth forest or on protected land. In this way, data

frames are to matrices what lists are to vectors. Because of this flexibility, data frames are one of the most common types of data structures in R.

Columns of a matrix or data frame should be the measured variables, and rows should be cases (localities, samples, etc.). This is a core principle of what is called *tidy data*, which plays an important role in many aspects of R.

Many of the ways used to create matrices will also work for data frames; the two kinds of objects are similar in most ways. A matrix can easily be converted to a data frame:

```
> x <- as.data.frame(x)
```

If all the values have the same type, a data frame can be similarly converted to a matrix.

```
> x <- as.matrix(x)
```

A data frame can also be built by combining vectors, but using the `data.frame()` command instead of `cbind()`. For example, suppose we had vectors for the locality name, the abundance of four different beetle species, and whether the forest is old-growth:

```
> locality <- c("Athens", "Comer", "Whitehall",  
               "Watkinsville", "Commerce")  
> beetle1 <- c(14, 23, 16, 19, 2)  
> beetle2 <- c(5, 1, 0, 0, 18)  
> beetle3 <- c(2, 3, 4, 2, 2)  
> beetle4 <- c(0, 0, 2, 3, 20)  
> isOldGrowth <- c(TRUE, TRUE, TRUE, TRUE, FALSE)  
> forests <- data.frame(locality, beetle1, beetle2, beetle3,  
                        beetle4, isOldGrowth)
```

It is good practice to remove those vectors after creating the data frame or matrix.

```
> remove(locality, beetle1, beetle2, beetle3, beetle4,  
         isOldGrowth)
```

Like matrices, it is easier to make large data frames in a spreadsheet and import them into R.

Row and column names are easily obtained with the `rownames()` and `colnames()`. The `names()` function is another way to get the column names.

```
> rownames(forests)  
> colnames(forests)  
> names(forests)
```

Column names can be created or changed by assigning a vector of names:

```
> colnames(forests) <- c('locality', 'S.andrewsi',  
                        'P.pyralis', 'E.pennsylvanica', 'C.bidenticola', 'oldGrowth')
```

The same can be done for rows.

Elements of a data frame are retrieved the same way as for a matrix, with the first index being the row and the second being the column.

```
> forests[3, 2]
```

A better and self-commenting approach is to specify the column by name.

```
> forests[, 'locality']
```

Even better, and far more widely used, is the `$` operator, which does not work for matrices.

```
> forests$locality
```

Dollar-sign notation cannot be used to select multiple columns. To select multiple columns, pass a vector of column names or numbers to select multiple columns.

```
> forests[, c('S.andrewsi', 'P.pyralis', 'E.pennsylvanica')]
> forests[, 2:4]
```

To access individual elements (cases) of a column in a data frame, use dollar-sign notation followed by brackets with indices, or specify the row and column numbers. Both of the following return the first four values (rows) of `locality` in column 1 of the data frame.

```
> forests$locality[1:4]
> forests[1:4, 1]
```

If the name of a data frame is long, dollar sign notation can become cumbersome. To avoid repeatedly typing the name of the data frame, you can use the `attach()` function, which creates copies of all the vectors (columns) in the data frame that can be directly accessed. When you are done with that data, it is good practice to use the `detach()` function to delete these copies.

```
> attach(forests)
> locality      # locality now accessible without $ notation

> detach(forests)
> locality      # returns an error; detach() destroyed the copy
```

The `attach()` and `detach()` commands also work on lists, but they do not work on matrices.

The `attach()` command comes with two main hazards, so it should be used with caution and only in limited situations.

First, because `attach()` creates copies, changes made to the attached data are not saved in the original data frame. Likewise, any changes made to the data frame after `attach()` is called are not reflected in the attached vectors.

Second, if `attach()` is called on two objects that have variables with the same name, a warning about “The following objects are masked” will be issued. Although both copies of the variable can be accessed, you can easily confuse which one you are using.

The **attach()** function is best used when working with a single data frame or list, and call **detach()** as soon as possible. There are alternatives to **attach()**, such as including the **data** parameter in some functions (like regressions), which lets you access variables of the data frame directly. You can also use the **with()** function, another safe alternative that can be used in many situations.

Sorting a matrix or data frame is more complicated than sorting a vector. It is often best to store the sorted matrix or data frame in another object in case you make a mistake while sorting. The following command sorts the data frame A by the values in its third column and stores the result in data frame B.

```
> B <- A[order(A[,3]), ]
```

To understand how any complicated command like this works, start at the innermost set of parentheses and brackets and work outward. At the innermost level, **A[,3]** returns all rows of column 3 from matrix A. Stepping out one level, **order(A[,3])** returns the order of the elements in that column as if they were sorted in ascending order. In other words, the smallest value would have a value of 1; the next smallest would have a value of 2, and so on. Finally, these values specify the desired order of the rows in the A matrix, and **A[order(A[,3]),]** returns all the columns with the rows in that order.

This technique of taking apart a complex function call is known as *unpacking*. You should do this whenever a complex function call returns unexpected results or you need to understand how a complex function call works.

Inspecting and converting objects

You will often want to inspect objects to see what they contain. Especially when objects are imported (described below), you will want to verify that they were imported correctly. Some functions work on only certain classes or types of objects, and when an error arises, the object should be inspected.

One approach for inspecting an object is simply to type its name to display its contents, but if the object is large, the result can be unreasonably voluminous. A much better approach is to use one or more of the five common functions for examining objects: **class()**, **head()**, **tail()**, **str()**, and **summary()**.

The **class()** function states whether an object is a list, matrix, data frame, or a function. If the object is a vector, it reports whether the values are integers, numeric, character, or logical.

The **head()** and **tail()** functions display the first few values of lists, vectors, data frames, and matrices. By supplying a numerical argument, you can control how many values are displayed.

```
head(x, 10)
```

In many cases, the **str()** function is a combination of **class()** and **head()**, making it very useful. For a data frame, it will report that it is a data frame, but also the number of rows

and columns, the name and type of every variable, and the first few values of every variable. For vectors, it will display the type, length, and first few values.

The **summary()** function will display some statistical information about the object. For vectors, matrices, and data frames, it reports the minimum, first quartile, median, mean, third quartile, and maximum. This makes it a great tool for checking for outliers and errors in the data.

For many operations, certain kinds of data structures are interchangeable. For example, many functions will accept matrices as well as data frames. Sometimes they won't, and you will need to convert one object type into another. R has many functions for doing this, and here are a few:

```
x <- as.matrix(x)
x <- as.data.frame(x)
x <- as.vector(x)
x <- as.list(x)
```

Importing Larger Data Sets

For larger data sets, it is easier to enter the data in a spreadsheet (like Excel), check it for errors, and export it into a format you can read into R. The most widely used file formats are comma-delimited and tab-delimited text. In Excel, choose Save As..., then select "Comma-separated values (.csv)" or "Tab-delimited text (.txt)." Both are text-only formats that can be read by almost any application.

It is good practice to have only a single file containing a data set, a practice known as the "single source of truth". If an error is fixed, additional data is added, or other changes are made, make those changes and store the file with the *same name*. Appending to the name is certain to lead to a confusing and ever-growing mess with variations that will be difficult to distinguish later on, such as myData.csv, myDataCleaned.csv, myDataCleaned2.csv, myData-Fixed.csv, etc. If you want to keep a backup of the original file, do so, but put it in a clearly marked directory like "archive", so that the old versions are in a different location from the working file. In addition, Excel is useful for editing files, but you should store the data in .csv files rather than having .csv files and .xls files that contain the same data. Another advantage to using .csv files only is that .csv files are plain-text files that can be read by many programs now and into the future. The same cannot be said of binary files like Excel.

To read or write files in R, you need to know the current working directory for R, that is, the directory in which R will look for files or save files. The **getwd()** function returns the current working directory.

```
> getwd()
```

It is easiest to use a single working directory during any given R session, and you can set this with the **setwd()** function. This directory should contain most or all of the files you will

need to read, such as data and source code, and it will be where files are saved by default. R will stay in this working directory unless you specify a different path to a file.

Paths to files and directories are specified differently in UNIX (macOS, Linux) and Windows file systems. This is one of the few areas where R works differently on the two platforms. If you share code across platforms, any command that accesses the file system will produce an error.

Use the `setwd()` function to set the working directory of R, with the path as an argument. Because the path is a string, you must wrap it in quotes, and this is true for any string in R. Single quotes and double quotes both work; just be consistent. Inside the path, use single forward slashes for UNIX systems:

```
> setwd('/Users/myUserName/Documents/Rdata')
```

Use double back-slashes for Windows:

```
> setwd('C:\\Documents and Settings\\myUserName\\Desktop')
```

Never include calls to `setwd()` in R code that you share with someone else. That path will likely not exist on their computer, and including it will produce errors when they run your code. Likewise, avoid embedding file paths within other commands.

IMPORTING A VECTOR

Once you have set your working directory, you can import a vector into R with the `scan()` function. Remember to put your file name in quotes and include the suffix.

```
> myData <- scan(file='myDataFile.txt')
```

This simple version of this command assumes that the cases are on different lines, but if cases are separated by commas, spaces, tabs, or other such characters, that can be set using the `sep` argument for `scan()`. Likewise, suppose the variable name is included in the file, or additional information is included at the beginning of the file. In that case, those lines can be skipped by setting the `skip` argument.

IMPORTING A DATA FRAME

Use the `read.table()` function to read a table of values into R and save it as a data frame. This assumes that your data are in the correct format, with columns corresponding to variables and rows corresponding to samples. All cells should contain a value; do not leave any empty. If a value is missing, enter **NA** in that cell. Variable (column) names can be as long as you need them to be, but you will want to strike a balance between readability and the amount of typing required. Variable names do not need to be single words, but any blank spaces will be replaced with a period when you import the data into R.

There are other commands for opening files, like `read.csv()`, but you should become comfortable opening any table file with `read.table()`; it is the Swiss Army knife of file openers.

Of the many options for `read.table()`, you will always pay close attention to three: the *header*, *row names*, and *separators*. You can evaluate these by examining the data file in a text editor. First, check whether column names exist for the variables; this row of column names is called a *header*. Second, check to see if there are *row names* (usually a sample number or locality name that is always unique to each row) and what column they are in. Row names are most commonly placed in the first column. Third, check what separates the columns: tabs or commas are the most common *separators*, also called *delimiters*.

To view these, open your file in a text editor like BBEdit or Notepad++ to see the actual structure of the file. Do not use word processors (Word, Pages, Google Docs) or spreadsheets (Excel, Numbers, Google Docs) because they convert the format of the file.

Suppose you have opened your table in a text editor and can see that it has the names of its variables in its first row (that is, it has a header), that it has sample names in the first column of the table, and that commas separate the values. Given this, you would import your data as follows:

```
> myData <- read.table(file='myDataFile.txt', header=TRUE,
  row.names=1, sep=',')
```

If there is no header, leave out the argument for the **header** parameter because the default is that there is no header. Similarly, omit the **row.names** parameter if there are no row names. If tabs separate the values, use **sep='\\t'**; if blank spaces separate the values, use **sep=' '**. If additional explanatory lines about the data are included at the beginning of the file, they can be skipped by supplying an argument to **skip**. For example, suppose reference information is supplied in the first two lines of the file, you can skip them like this:

```
> myData <- read.table(file='myDataFile.txt', header=TRUE,
  row.names=1, sep=',', skip=2)
```

There are over twenty other parameters for `read.table()`; it truly is a Swiss Army knife, and I have never encountered a file I could not open with it. Consult the help page if you are unable to open a file, as one of these parameters is likely the key.

Always verify that an imported data file was read correctly, and `str()` is probably the most helpful. Depending on what you need to see, `class()`, `head()`, `tail()`, and `summary()` may help.

Generating Numbers

Writing long, regular sequences of numbers is tedious and error-prone. R can generate these quickly with the `seq()` function. If the increment (`by`) isn't set, it defaults to 1.

```
> seq(from=1, to=100, by=2)      # odd numbers from 1 to 100
> seq(from=0, to=100, by=5)      # counting by fives
> seq(from=1, to=100)            # integers from 1 to 100
```

You can repeat a value with the `rep()` function; the first argument is the value to be repeated, and the second is the number of repeats:

```
> rep("Gulf of Mexico", 47)
> rep("I will not waste chalk", 200)
```

R can generate random numbers from many statistical distributions. A few common examples include `runif()` for uniform or flat distributions, `rnorm()` for normal distributions, and `rexp()` for exponential distributions. For each, specify the number of random numbers (`n`) and one or more parameters that describe the distribution. For example, this generates ten random numbers from a uniform distribution that starts at one and ends at 6:

```
> runif(n=10, min=1, max=6)
```

By rounding the result, this could be modified to simulate the rolls on a fair 6-sided die.

```
> dice <- round(runif(n=100000, min=0.5, max=6.5))
```

Random numbers from many distributions can be created, including normal, log-normal, and exponential:

```
> rnorm(n=100, mean=5.2, sd=0.7) # custom shape parameters
> rlnorm(n=1000)                  # default shape parameters
> rexp(n=50, rate=7)              # custom shape parameters
```

Basic Statistics

R has functions for calculating simple descriptive statistics from a vector of data, such as:

```
> mean(x)      # arithmetic mean
> median(x)    # middle value
> length(x)    # number of elements in x (sample size, or n)
> range(x)     # largest and smallest value
> sd(x)        # standard deviation
> var(x)       # variance
> summary(x)   # minimum, maximum, mean, median, and quartiles
```

Many statistical tests are built in. Some common ones include:

```
> t.test(x, y)           # t-test for equality of means
> var.test(x, y)         # F-test for equality of variance
> cor(x, y)              # correlation coefficient
> lm(y~x)                # least squares regression
> anova(lm(x~y))         # ANOVA (analysis of variance)
> wilcox.test(x, y)      # Mann-Whitney U test
> kruskal.test(x~y)      # Non-parametric ANOVA
> ks.test(x, y)         # Kolmogorov-Smirnov test
```

In addition to generating random numbers, the built-in statistical distributions of R can be used to find probabilities or p -values, find critical values, and draw probability density functions. These commands are set up consistently, although their parameters may differ. For example, these can be performed with the normal distribution through the `pnorm()`, `qnorm()`, and `dnorm()` functions.

The `pnorm()` function will find the area under a standard normal curve, starting from the left tail and going to a value of 1.96 in the example below. Note that one minus this value would correspond to the p -value for a Z -test, where $Z=1.96$:

```
> pnorm(q=1.96)
```

The `qnorm()` function can find the critical value for a given probability. For example, in a two-tailed test at a significance level of 0.05, the following generates the critical value on the right tail.

```
> qnorm(p=0.975)
```

Last, the `dnorm()` function gives the density or height of the distribution at a particular value (at 1.5 below). By calculating this for every value, you can plot the shape of the distribution.

```
> dnorm(x=1.5)
```

Other distributions, such as the uniform, exponential, and binomial distributions, follow a similar syntax, with `r` corresponding to a random number, `p` for probability, `q` for critical value, and `d` for density (the height of the function at a particular value).

```
> rbinom(n=10, size=15, prob=0.1) # binomial distribution
> pexp(q=7.2)                     # exponential distribution
> qunif(p=0.95)                   # uniform distribution
> dlnorm(10)                      # log-normal distribution
```

One of R's great strengths is the large number of available statistical analyses. Many are built into the base package of R, and a mind-boggling number is freely available in user-contributed packages. There's a good chance that if you need to do an analysis, it is already available in R. Installing packages is described under Customizing R below.

Efficient computation

R's vectorized math eliminates the need for looping through vectors and matrices to perform many calculations. Such vectorized calculations are commonly orders of magnitude faster than using loops.

An important set of tools for efficient computation is the **apply()** family of functions. The most basic of these is **apply()**, used when you want to perform an operation on every row or every column of a matrix or data frame and get the results as a vector. For example, the following calculates the median for every column and returns a vector with those medians.

```
> medians <- apply(x, MARGIN=2, FUN=median)
```

Set **MARGIN** to 1 to calculate the median for every row instead. The easy way to remember whether to use 1 or 2 is to consider how rows and columns are specified for matrices and data frames: it is [row, column], so rows are 1, and columns are 2. The argument to **FUN** is the name of the function to be applied; this could be a function you create.

The functions **sapply()** and **lapply()** are similar but are used when a function needs to be applied to all elements of a vector or a list. **lapply()** returns a list, and **sapply()** returns a vector or matrix. One possible use of both would be to store a series of similar data sets as elements of a list, then call **lapply()** or **sapply()** once to perform some function on each data set. This would be simpler than looping through the list and performing the calculations separately on each data set. It is also better than storing the data sets as separate objects and performing the calculations separately on each object. By creating your function as a wrapper around other functions, you could perform complicated multi-step analyses, including making and saving plots for every data set in one command.

Plots

R has excellent plotting capabilities. Many types of plots are available, and complex plots can be produced. R's plots are far superior to those made in Excel or Numbers, which commonly have unnecessary shading, grid lines, and 3-D effects. The plots described below are part of the base R package that is standard to every distribution of R, and the **ggplot2** package offers many other ways to plot data; see Learning R (above) for good guides to **ggplot**.

When calling an R plotting function, the plot will automatically be generated in the active plotting window if one already exists. If no plot window exists, a new one will be created. Use **dev.new()** to create a new plot window, as this command will work on any platform. Avoid using platform-specific ways of creating new plot windows, such as **quartz()** for macOS, **X11()** for Linux, and **windows()** for Windows, because these will generate errors if your code is run on a different platform.

The height and width of a plot window (in inches) can be specified, or they can be left off to create the default window.

```
> dev.new(height=7, width=12) # custom size, in inches
> dev.new()                  # default size (7 x 7 on macOS)
```

HISTOGRAMS

Histograms, also called frequency distributions, are generated with the **hist()** function. Calling **hist()** with the data as the only argument makes a default histogram.

```
> hist(x)
```

The number of bars or divisions in the distribution is set with the **breaks** parameter.

```
> hist(x, breaks=20)
```

Oddly, specifying breaks this way is only a suggestion, and it may not be honored. Specific breaks can be forced by supplying a vector of breaks that encompasses all values in the data.

```
> divisions <- c(-2.5, -2, -1, 0, 1, 1.5, 2, 2.5)
> hist(x, breaks=divisions)
```

SCATTERPLOTS

Scatterplots (x-y plots) are generated with the **plot()** function, with **x** on the horizontal axis and **y** on the vertical axis.

```
> plot(x=vegetationDensity, y=beetleAbundance)
```

Calling these arguments by position is common, putting the **x** first and **y** second.

```
> plot(vegetationDensity, beetleAbundance)
```

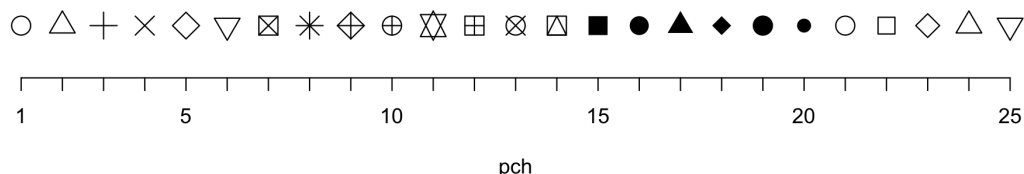
Labels for the x and y axes can be set with the **xlab** and **ylab** parameters, and the plot title can be set with the **main** parameter. Remember to put strings in quotes.

```
> plot(Fe, pH, main='Acid mine drainage', xlab='Fe (ppt)',
      ylab='pH')
```

The **pch** parameter can be set to specify a particular type of symbol. The value of 16 is a filled circle, an easily visible symbol that you should use. For brevity, I will use **x** and **y** for the following examples, but you should always use informative names for your objects, like **pH** or **percentQuartz**.

```
> plot(x, y, pch=16)
```

Twenty-five plotting symbols can be specified with **pch**:



Changing the value of **type** controls whether lines or points are plotted.

```
> plot(x, y, type='p') # points only
> plot(x, y, type='l') # connecting lines only, no points
> plot(x, y, type='b') # both points and lines
> plot(x, y, type='n') # no points, only axes and labels
```

Symbol colors can be set with the **col** parameter. To see the full list of 657 named colors in R, use the **colors()** function. Even better is the guide provided by R Charts at <https://r-charts.com/colors/>. Colors can also be specified by RGB (red-green-blue) value, as used in creating colors for web pages.

```
> plot(x, y, pch=16, col='green') # small green filled circles
> colors()                        # lists all named colors
> plot(x, y, col='#00CCFF')       # 00CCFF is pale blue
```

Equations and special symbols can be used in figure labels and as text added to figures. See the help page for **plotmath()** for more details. Adding equations and special symbols commonly uses three functions. The function **text()** places additional text onto a figure, **expression()** creates an object of type “expression,” and **paste()** is used to combine items into a single string. The help pages for these functions are useful for seeing the primary ways of combining these functions. The first example below puts the square root of x as the y -axis label, and the second shows the standard symbol for the oxygen isotope ratio ($\delta^{18}\text{O}$) on the y -axis.

```
> plot(x, y, ylab=expression(paste(sqrt(x))))
> plot(x, y, ylab=expression(paste(delta^18, 'O')))
```

Special symbols can be added using Unicode symbols in **expression()** and **paste()**. For example, to create a label with a per-mil symbol, such as $\delta^{13}\text{C}$ (‰ PDB), use the Unicode character 2030; all Unicode character numbers must be preceded with **\U**.

```
> expression(paste(delta^13, 'C (\U2030 PDB)'))
```

Symbols for thousands of other characters can be found at <https://home.unicode.org>. You can even plot emojis.

ADDING TO PLOTS

Labels and annotations can be added with the `text()` function, where the `x` and `y` arguments specify the coordinates on the plot, and the `labels` argument specifies what text will be added. The coordinates are identical to those used for placing points on the plot.

```
> text(x=10, y=2, labels=expression(sqrt(x)))
```

As these are the first three arguments to `text()`, this can be called more simply as

```
> text(10, 2, expression(sqrt(x)))
```

Lines can be added to a plot with the `abline()` function. You can make horizontal lines, vertical lines, and lines with a specified slope and intercept.

```
> abline(h=5)                # horizontal line at y=5
> abline(v=10)               # vertical line at x=10
> abline(a=intercept, b=slope) # line with slope and intercept
> abline(lm(y~x))            # fitted regression line
```

Geometric shapes can be added to a plot, and they can be customized with line types (dashed, dotted, etc.), line colors, and fill colors. See `lty` in `par()` for customizing line types. Become familiar with the help page for `par()`; it describes how plots can be customized.

The `rect()` function draws a box with the coordinates of the left, bottom, right, and top edges.

```
> rect(xleft=2, ybottom=5, xright=3, ytop=7)
```

The `arrows()` function draws an arrow, with the first two arguments specifying the `x` and `y` coordinates of the line's origin. The third and fourth arguments specify the coordinates of the arrowhead tip.

```
> arrows(x0=1, y0=1, x1=3, y1=4)
```

The `segments()` function is identical to the `arrows()` function, except that no arrowhead is drawn.

```
> segments(x0=1, y0=1, x1=3, y1=4)
```

Similarly, lines connecting data points can be added with `lines()`, and polygons can be added with `polygon()`. A box can be drawn around the entire plot with `box()`.

BUILDING A COMPLEX PLOT

Complex plots are typically built in stages, with groups of points added sequentially and custom axes, text, and graphic annotations added separately. To construct a plot in stages, set **type='n'** to prevent data points from being plotted, and set **axes=FALSE** to prevent the axes from being drawn. Specify the values to be plotted along the x and y axes as the first two arguments.

```
> plot(x, y, type='n', axes=FALSE)
```

This will produce a plot window with only the x and y-axis names: no points, ticks, boxes, axis labels, etc. Points can be added to the current plot with the **points()** function. Only those points that are visible within the present limits of the axes will be shown. The limits of the axes are based on the values in the data supplied to the **plot()** function.

```
> points(x, y)
```

Axes can be added individually with the **axis()** function. In this example, the initial argument specifies the axis to be drawn, with 1 indicating the x-axis and 2 indicating the y-axis. The **at** parameter sets where the tick marks should be drawn, and the **labels** parameter specifies the label to be displayed next to each tick mark. Values given to **labels** must be in quotes, as they are strings.

```
> axis(1, at=seq(-1.0, 2.0, by=0.5), labels=c('-1.0', '-0.5',  
      '0.0', '0.5', '1.0', '1.5', '2.0'))
```

Here is an example of building a complex plot in parts. The commands I show were preceded by experimentation (not shown) with different parts of the plot to determine the order in which these commands would be given. The data are read in, and **attach()** makes the variables accessible without dollar-sign notation.

```
> waves <- read.table(file='waves2.txt', header=TRUE, sep=',')  
> attach(waves)
```

Plotting begins by making a window with an aspect ratio appropriate for the data. This is a time series, so the plot should be relatively wide to emphasize the changes over time.

```
> dev.new(height=4, width=10)
```

Because this is a scatterplot, the basic plot is built with the `plot()` command, specifying the x and y data as the first two arguments, the titles, the axis labels, and the limits of the y data range (`ylim`). The type is set to `n` because the data will be added subsequently.

```
> plot(x=juliandate, y=SwH, main='Station 46042 - Monterey -  
27 NM West of Monterey Bay, California', xlab='',  
ylab='Significant Wave Height (m)', axes=FALSE,  
type='n', ylim=c(0, 6))
```

A light gray rectangle is added to emphasize a particular region. The call to `plot()` specified `type='n'` so that the rectangle can be added and not cover the data. The `gray()` command sets the color to a light shade of gray, with 0.0 being black and 1.0 being white.

```
> rect(12, 0, 18, 6, col=gray(0.9), border=gray(0.9))
```

Two lines are added to emphasize divisions of the data along the x-axis. Note that a vector of two values is passed as the `v` argument, which is simpler than drawing the lines with separate calls to `abline()`.

```
> abline(v=c(1, 32), col=gray(0.5))
```

Because there are so many data points, the data are added as a series of lines (`type='l'`) that connect individual values rather than showing a symbol for each data point. Date (`juliandate`) is plotted along the x-axis, and wave height (`SwH`) is plotted along the y-axis, matching the arguments in the `plot()` call.

```
> points(juliandate, SwH, type='l')
```

The x-axis is built in two stages. Closely spaced and unlabelled short ticks are added first, with `tcl` specifying the tick length.

```
> axis(1, at=seq(0, 46, by=1), labels=FALSE, tcl=-0.3)
```

Widely spaced longer ticks are added next, and these are labeled:

```
> axis(1, at=seq(0, 45, by=5), labels=c('30 Nov',  
'5 Dec', '10', '15', '20', '25', '30', '4 Jan', '9', '14'))
```

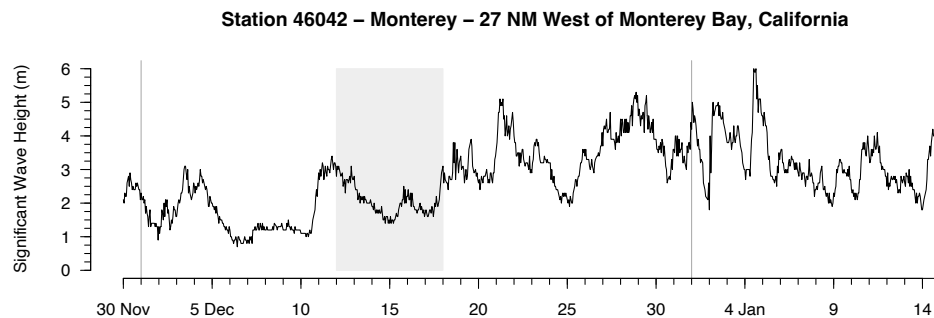
Likewise, the y-axis is built in two stages, beginning with closely spaced unlabeled short ticks:

```
> axis(2, at=seq(0, 6, by=0.25), labels=FALSE, tcl=-0.3)
```

These are followed by widely spaced long ticks that have labels:

```
> axis(2, at=seq(0, 6, by=1), labels=  
c('0', '1', '2', '3', '4', '5', '6'), las=1)
```

This is the final plot. The advantage of making the plot this way, instead of making a simple plot and modifying it in Illustrator, is that if you make any mistakes, you can fix them, re-run the commands, and have your final plot. This saves you from laboriously making all the edits in Illustrator again. Every R plot can be saved as a PDF file, which can be edited in Illustrator.



M U L T I P L E P L O T S O N A P A G E

Multiple plots can be drawn on the same page by using the `par()` function and setting the argument to `mfrow`. Each successive call to `plot()` will fill the panel from left to right or from top to bottom, starting in the upper left corner.

```
> par(mfrow=c(3, 2))
```

This example will produce a page with areas for six plots in 3 rows and two columns. Each plot is added subsequently. Note that a plot cannot be modified once another plot has been created.

Saving and Exporting Results

C O M M A N D S

You can copy and paste the entire R session into a text file from the R console window. This will save a record of your commands and their displayed results. This is an easy way to save all your work, the commands, and the results. Making a habit of this allows you to replicate your work, build on it subsequently, and apply your analyses to other data sets.

You can edit this text file to contain only the commands you need, which makes it easy to retrace your steps or re-run your commands if you discover an error. First, delete all the commands and their output that produced results you don't want to keep. Next, delete all lines of results, leaving only lines of commands. Last, delete the `>` prompts at the beginning of each line and any `+` marks at the beginnings of lines. If you copy and paste this code into a

new R session, it should run without errors or warnings. If you get errors or warnings, edit the code and repeat this process until it runs clean. Save this file for a permanent record of your analysis; it should be a plain-text file with a .R suffix. Always pick a file name that conveys the purpose of the code, such as `diversityAnalysis.R` or `isotopicPlots.R`.

PLOTS

On macOS, graphics produced in R can be saved as PDF files by selecting the plot's window and choosing Save or Save As from the File menu. In Windows, graphics can be saved this way to metafile, postscript, PDF, .png, .bmp, and .jpeg formats. The PDF format is recommended because it can be edited in Adobe Illustrator. Raster (bitmap) formats such as .png, .bmp, and .jpeg should be avoided because they cannot be edited and often suffer from compression artifacts that make them look fuzzy.

Graphics can also be built and saved to a file from the command line. Use the `pdf()` function to open a PDF file, just as you would open a plotting window with `dev.new()`. Run all your plotting commands, followed by `dev.off()` to close the PDF file. Note that your plot will not be displayed while you do this, although it is being written to a file.

```
> pdf(file='myPlot.pdf', height=7, width=7)
> plot(x, y)
> # ... all other plotting commands
> dev.off()
```

A typical strategy is to first develop a plot onscreen to refine the order of the plot commands. Once the plot commands are established, open the file with `pdf()`, rerun the plotting commands, and close the file with `dev.off()`.

OBJECTS

Any objects generated in R, such as functions, vectors, lists, matrices, data frames, etc., can be saved to a file using the command `save.image()`. Although this will not save any graphics or the command history, it preserves the objects produced by your analyses so they will not have to be regenerated in future R sessions. I recommend deleting unnecessary objects before saving the image, especially temporary or test objects generated when you experiment with code. If your work involves long-running computations to build an object, use `save.image()` so you don't have to re-run those analyses the next time you work. The image should have the suffix `.RData`.

```
> save.image(file = 'myResults.RData')
```

When you resume your work, reload those objects with `load()`, allowing you to continue where you left off.

```
> load(file = 'myResults.RData')
```

Programming in R

If you have previous programming experience, read *The Art of R Programming*; it will save you much time and will help you connect the R way of doing things to what you already know.

When coming from other programming languages, one of the most significant conceptual hurdles is learning to avoid loops, which can dramatically slow your calculations. Nested loops in R can be brutally slow. You can avoid loops through vectorized operations and the **apply()** family of functions.

Even so, some problems require loops. Loops can also make code more understandable in some cases.

R's **for** loop is generalized like the *do* loop in FORTRAN and the *for* loop in C, Python, and MATLAB. If you have programmed before, the syntax is intuitive, and the **for** loop will likely be the most common loop that you use. Use curly braces if there is more than one statement inside the loop. There are various styles as to where the curly braces may be placed to help the readability of the code, but their placement does not affect the execution. Indenting the code inside the loop helps to make the contents of the loop more apparent. Although indenting isn't required, it clarifies your code and makes it easy to parse.

Here is an example of calculating variance with loops, as you might in C or Fortran.

```
> x <- rnorm(10000000)    # simulating a very large data set

> sum <- 0
> for (i in 1:length(x)) sum <- sum + x[i]
> mean <- sum/length(x)    # mean() would be faster
> sumsquares <- 0
> for (i in 1:length(x)) {
+   sumsquares <- sumsquares + (x[i]-mean)^2
+ }
> variance <- sumsquares / (length(x)-1)
```

On a recent computer, this ran in 16.3 seconds. When vectorized math was used instead of loops,

```
> variance <- sum((x-mean)^2) / (length(x)-1)
```

it ran in a mere 0.23 seconds, a dramatic improvement (70 times faster). Using the built-in **var()** function, which has additional optimizations, it took only 0.082 seconds (almost 200 times faster). These three times were 0.43, 0.06, and 0.05 seconds on a newer computer.

R has other typical flow-control statements, including **if** and **if-else** branching, **while** and **repeat** loops, **break** statements, and so on.

The `if` test will let you run one or more statements if some condition is true. Enclose the logical tests in parentheses. If only one line of code is executed for the test, it can be placed on the same line.

```
> if (x < 3) y <- a + b
```

If multiple lines of code are to be executed, enclose them in curly braces. There is an `if-else` test that can let you act on a variety of conditions.

```
> if (x < 3) {  
  y <- a + b  
  z <- sqrt(x)  
} else {  
  y <- a - b  
  z <- log(a + b)  
}
```

The `while` loop will execute as long as the condition is true.

```
> while(x >= 0) { some expressions }
```

Note that if this condition never becomes false, this will generate an infinite loop.

TIMING YOUR CODE

Critical to optimizing your code is timing it. To do this, add one line before the code you wish to measure:

```
> startTime <- Sys.time()
```

Add these two lines after the code to be measured:

```
> endTime <- Sys.time()  
> endTime - startTime
```

Copy all of these commands (the line above, your code, the two lines above) into R in one step. When your code executes, it will report how long it took.

Always measure your code before you optimize it to understand the effects of your optimizations. In some cases, code that seems like it ought to be faster might not be. Sometimes a loop is faster than the alternative. Be aware that some R users are dogmatic purists about avoiding R loops. Be pragmatic, not dogmatic: measure your code and use the fastest or most direct solution.

Customizing R

PACKAGES

A great advantage of the open-source nature of R is that users have contributed an astonishing number of packages for solving a vast array of data analysis problems. For example, packages have been developed explicitly for non-parametric statistics, signal processing, and ecological analyses. Using these packages requires two steps.

First, they must be *installed* on your system, which is a one-time step. Most packages can be installed from the R site: <http://CRAN.R-project.org/>. Packages can be installed on macOS through the package installer under the Packages & Data menu. To check which packages have been installed and are available to be loaded, use the **library()** function.

```
> library()
```

Second, once a package is installed, it should be *loaded* by calling the **library()** function with the name of the library as an argument. A package is an object, not a string, so do not surround it with quotes. For example, this is how you would load the **vegan** package, a valuable package for ecological analyses.

```
> library(vegan)
```

Using the **library()** function allows you to call the functions in a library easily. Without it, you would need to call the function and the package name separated by a pair of colons. Calling the **decostand()** function of the **vegan** package would be done like this:

```
> vegan::decostand(...)
```

This is often undesirable, so packages are commonly first loaded with a **library()**. Be aware that the correctness of code in packages may not have been thoroughly tested. Use trusted packages rather than obscure ones, and verify your results.

Packages are periodically updated, just as R is, so check to ensure that R and your packages are current. To check on the status of your packages, run **packageStatus()**, and to update your packages, run **update.packages()** and follow the prompts.

FUNCTION FILES

As you become familiar with R, you will write your own functions. To easily access these, save your functions to a text file. These can be loaded into an R session with the **source()** function. The .R file suffix can be lower-case or upper-case, although the convention is increasingly to prefer upper case.

```
> source('myfunctions.R')
```

I usually create one source file for each project, more for complex projects, and **source()** lets these functions be loaded in one step rather than copying and pasting the code.

If you find yourself typing the same commands at the beginning or end of every R session, you can set up R to automatically execute those commands whenever it starts or quits by editing your `.First()` and `.Last()` functions. Common commands to place in the `.First()` function include loading any commonly used libraries or data sets, setting the working directory, setting preferred graphics arguments, and loading any files of R functions. Here is an example of a `.First()` function:

```
.First <- function() {  
  library(utils)  
  library(grDevices)  
  library(graphics)  
  library(stats)  
  library(vegan)  
  library(MASS)  
  library(cluster)  
  setwd('/Users/myUserName/Documents/Rdata')  
  source('myfunctions.R')  
}
```

This function loads a series of libraries routinely needed for analyses, sets the working directory, and loads a set of functions written by the user.

On macOS and other UNIX/LINUX systems, the `.First()` and `.Last()` functions are stored in a file called `.Rprofile`, an invisible file located at the root level of a user's home directory (for example, `/Users/username/`). On Windows, these functions are stored in `Rprofile.site`, which is kept in the `C:\Program Files\R\R-n.n.n\etc` directory. If the `.Rprofile` or `Rprofile.site` file doesn't exist on your system, you will need to create one as a plain text file using a text editor. On macOS, Linux, and other UNIX-based systems, be sure to include the period at the beginning of the file name.